Transformers y Aplicaciones de Deep Learning

Día 3: Atención, modelos preentrenados y despliegue

Curso introductorio para TICs y Seguridad

Dr. Rubén Hernández Torres

21 de octubre de 2026

Objetivos de aprendizaje

- Comprender las limitaciones de las RNN.
- Explicar el principio de self-attention.
- Identificar los componentes de un Transformer.
- Comprender el aprendizaje por transferencia.
- Guardar y cargar modelos de PyTorch.
- Integrar un modelo en una aplicación.

Limitaciones de las redes recurrentes

En una RNN:

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$$

El cálculo del estado actual depende del estado anterior.

- Procesamiento secuencial.
- Paralelización temporal limitada.
- Dificultad con dependencias largas.
- Posibles problemas con los gradientes.

Pregunta

¿Es posible relacionar tokens sin procesarlos obligatoriamente uno después de otro?

La idea de atención

Los mecanismos de atención permiten combinar información de distintas posiciones.

Ejemplo:

```
Your account was suspended.
Click here to verify it.
```

Para interpretar el token `it`, puede ser útil considerar otros tokens del mensaje.

Self-attention

Cada posición puede utilizar información de otras posiciones de la misma secuencia.

Query, Key y Value

A partir de una representación de entrada X, se calculan tres proyecciones:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

- Query: información que consulta una posición.
- Key: información utilizada para comparar.
- Value: contenido que se combina.

Las matrices de proyección contienen parámetros aprendibles.

Scaled Dot-Product Attention

La operación fundamental es:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Interpretación:

1. Calcular similitudes entre posiciones.
2. Escalar los productos.
3. Normalizar mediante softmax.
4. Combinar los vectores Value.

Ejemplo conceptual de atención

Supongamos tres tokens:

$$[\text{verify}, \text{your}, \text{account}]$$

Una matriz hipotética de pesos:

$$A = \begin{bmatrix} 0{,}2 & 0{,}1 & 0{,}7 \\ 0{,}3 & 0{,}4 & 0{,}3 \\ 0{,}6 & 0{,}1 & 0{,}3 \end{bmatrix}$$

Cada fila indica cuánto peso asigna una posición a las demás.

$$Z = AV$$

Pregunta: ¿Por qué cada fila de esta matriz suma uno?

£Qué es un Transformer?

Un Transformer combina mecanismos de atención con otras transformaciones.
Componentes fundamentales:

- Embeddings.
- Información posicional.
- Multi-head attention.
- Redes feed-forward.
- Conexiones residuales.
- Normalización.

Los Transformers pueden utilizarse en tareas de clasificación, generación y representación.

Multi-Head Attention

En lugar de calcular una única atención, se utilizan varias cabezas.

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

Las salidas se concatenan y se proyectan:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W_O$$

Las diferentes cabezas pueden aprender patrones complementarios.

Información posicional

La self-attention sin información posicional no representa por sí misma el orden de los tokens. Por ello se incorporan representaciones de posición.

$$Z = X_{\text{emb}} + P$$

Donde:

- X_{emb}: embeddings de tokens.
- P: información de posición.

Existen diferentes métodos de codificación y representación posicional.

¿Qué es el preentrenamiento?

Entrenar modelos grandes desde cero requiere cantidades considerables de datos y recursos computacionales.
El preentrenamiento permite aprender representaciones reutilizables.

Transfer Learning

Aprovechar parámetros aprendidos en una tarea o conjunto de datos para resolver otro problema.

- Extracción de características.
- Fine-tuning.
- Inferencia directa.

BERT y GPT

BERT

- Arquitectura basada en encoder.
- Representaciones bidireccionales.
- Útil para comprensión y clasificación.

GPT

- Arquitectura basada en decoder.
- Atención causal.
- Generación autoregresiva.

Ambos pertenecen a la familia de modelos basados en Transformers.

Aplicaciones en ciberseguridad

Los modelos de texto pueden adaptarse a:

- Clasificación de correos sospechosos.
- Análisis de reportes de incidentes.
- Clasificación de logs.
- Análisis de descripciones de amenazas.
- Organización de documentación técnica.

Precaución

Un modelo de sentimientos no es automáticamente un detector de phishing. La tarea requiere datos y evaluación adecuados.

Entrenamiento frente a inferencia

Entrenamiento

- Se utilizan datos y etiquetas.
- Se calcula una pérdida.
- Se calculan gradientes.
- Se actualizan parámetros.

Inferencia

- Se reciben nuevas entradas.
- Se aplica el preprocesamiento.
- Se ejecuta el modelo.
- Se obtiene una predicción.

Durante la inferencia habitual no se actualizan los parámetros.

Guardar un modelo PyTorch

PyTorch permite guardar los parámetros entrenados.

```
import torch

torch.save(
model.state_dict(),
"modelo.pth"
)
```

Importante

El archivo contiene los parámetros, pero no sustituye la definición de la arquitectura ni el preprocesamiento.

Cargar un modelo

```
model = MLP()

state = torch.load(
"modelo.pth",
map_location="cpu",
weights_only=True
)

model.load_state_dict(state)
model.eval()
```

La arquitectura debe coincidir con la utilizada al guardar los parámetros.

Realizar inferencia

```
model.eval()

with torch.inference_mode():
logits = model(x)
predicted = logits.argmax(dim=1)

print(predicted)
```

- `eval()` activa el modo evaluación.
- `inference_mode()` evita el seguimiento de gradientes.
- El preprocesamiento debe ser consistente.

De un modelo a una aplicación

Una aplicación de Deep Learning necesita:

1. Recibir información del usuario.
2. Validar y preprocesar la entrada.
3. Convertirla en tensores.
4. Ejecutar la inferencia.
5. Interpretar la salida.
6. Presentar el resultado.

Herramientas

Gradio, FastAPI, servicios web y aplicaciones de escritorio.

Consideraciones de seguridad

Una aplicación real debe considerar:

- Validación de entradas.
- Protección de datos sensibles.
- Modelos y archivos de origen confiable.
- Cambios en la distribución de datos.
- Falsos positivos y falsos negativos.
- Supervisión y actualización del modelo.

Importante

Una predicción no constituye por sí sola evidencia definitiva de un ataque.

Otras arquitecturas de Deep Learning

Arquitectura	**Aplicaciones**
MLP	Datos tabulares y clasificación.
CNN	Imágenes y patrones espaciales.
RNN / GRU	Logs y secuencias temporales.
Autoencoder	Reconstrucción y anomalías.
Transformer	Texto y secuencias complejas.
GNN	Redes y estructuras de grafos.

La arquitectura adecuada depende del problema, los datos y los recursos.

Conclusiones del curso

- Las redes neuronales aprenden representaciones a partir de datos.
- PyTorch permite definir y entrenar modelos.
- Los embeddings representan información simbólica mediante vectores.
- Las GRU modelan dependencias secuenciales.
- Los Transformers utilizan atención.
- El despliegue conecta los modelos con aplicaciones reales.

Mensaje final

Deep Learning es una herramienta que debe seleccionarse y evaluarse según las necesidades del problema.